



Join us at javatechcommunity.com for our Premium Java Interview Kit and get referrals for Java-based roles – absolutely FREE!



WHAT IS HIBERNATE, AND WHAT ARE ITS USES?



What is Hibernate?

- Hibernate is a ORM tool for Java
- Using Hibernate, we map our Java objects with database tables.
- Hibernate simplifies database interactions in Java applications.

Advantages of Hibernate over JDBC

1. **Cache Support:** Supports two levels of caching
2. **Supports HQL:** Simplifies SQL usage
3. **Transaction Management:** Handles transactions efficiently
4. **Hibernate Supports Pagination:** Helps in managing large data sets efficiently
5. **Supports Different Relationships and Fetching Strategies:** Handles one-to-one, one-to-many, and many-to-many relationships efficiently



⚠ Copyright Notice:

This document is strictly for personal use only and not for redistribution or commercial purposes.

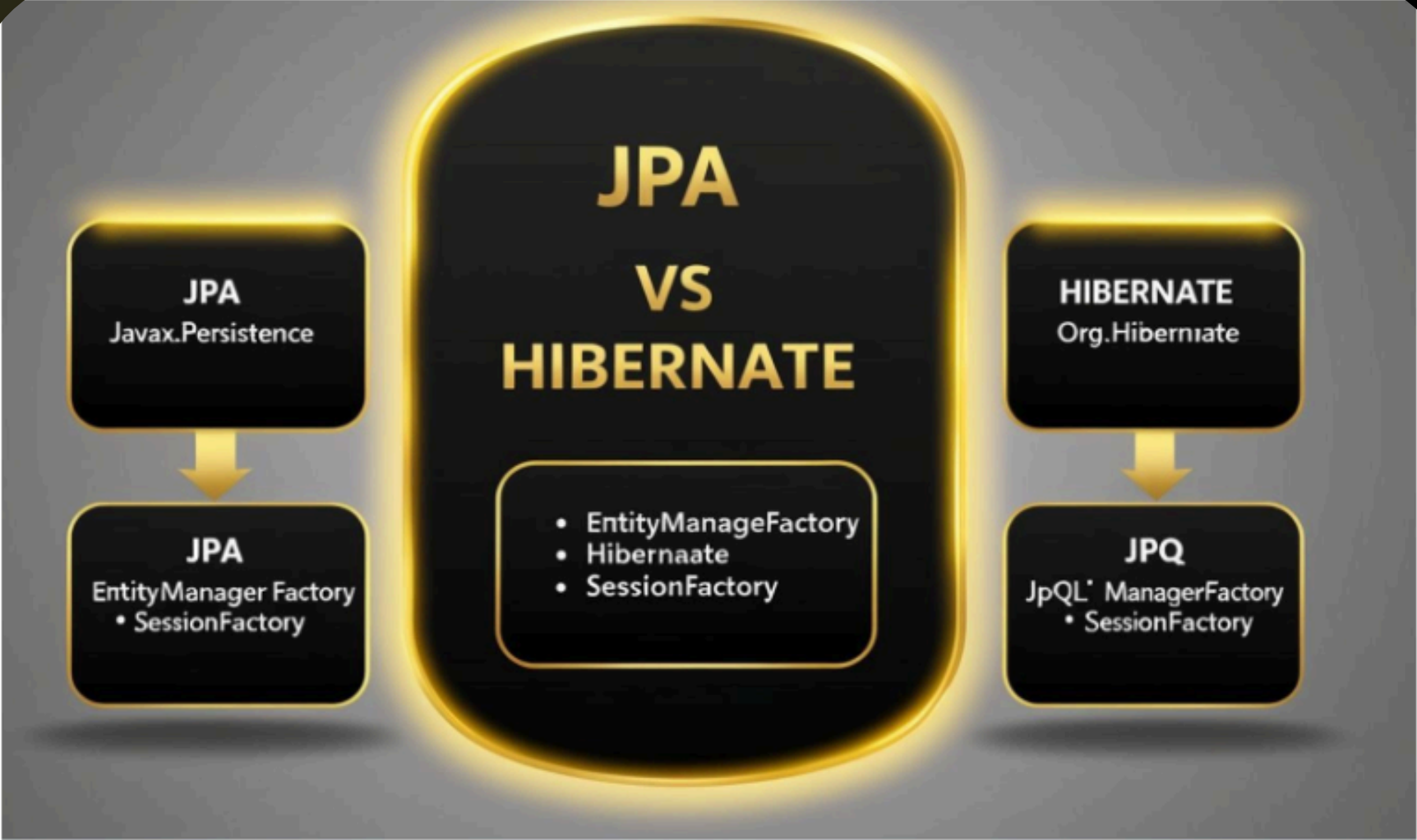
If you wish to use any part of this document, you must credit the source and creator: [@javatechcommunity.com](https://javatechcommunity.com) - support@javatechcommunity.com



Join us at javatechcommunity.com for our Premium Java Interview Kit and get referrals for Java-based roles – absolutely FREE!



EXPLAIN THE DIFFERENCES BETWEEN JPA AND HIBERNATE.



JPA vs Hibernate Comparison

Feature 📦	JPA 💎	Hibernate 💎
📦 Package	javax.persistence.JPA	org.hibernate
📖 Specification vs Implementation	Specification for connecting an application with a database	Implementation of JPA
⚙️ Entity Manager Interaction	EntityManagerFactory	SessionFactory
🔧 Performing CRUD Operations	EntityManager	Session Interface
🔍 Query Handling	JPQL	HQL

Additional Features of Hibernate Over JPA

- ✅ **Caching:** Improves performance by reducing database access.
- ✅ **Lazy Loading:** Reduces the amount of data loaded into memory, improving performance.
- ✅ **Criteria API:** A type-safe and fluent API for building queries.
- ✅ **Native SQL Queries:** Useful for complex queries or non-standard SQL features.
- ✅ **saveOrUpdate() Method:** Helps in handling insert or update operations efficiently.
- ✅ **@Immutable Annotation:** Marks an entity as immutable, meaning its state cannot be modified once persisted.
- ✅ **@BatchSize Annotation:** Specifies batch sizes for loading associations, improving performance by reducing database queries.



⚠️ Copyright Notice:

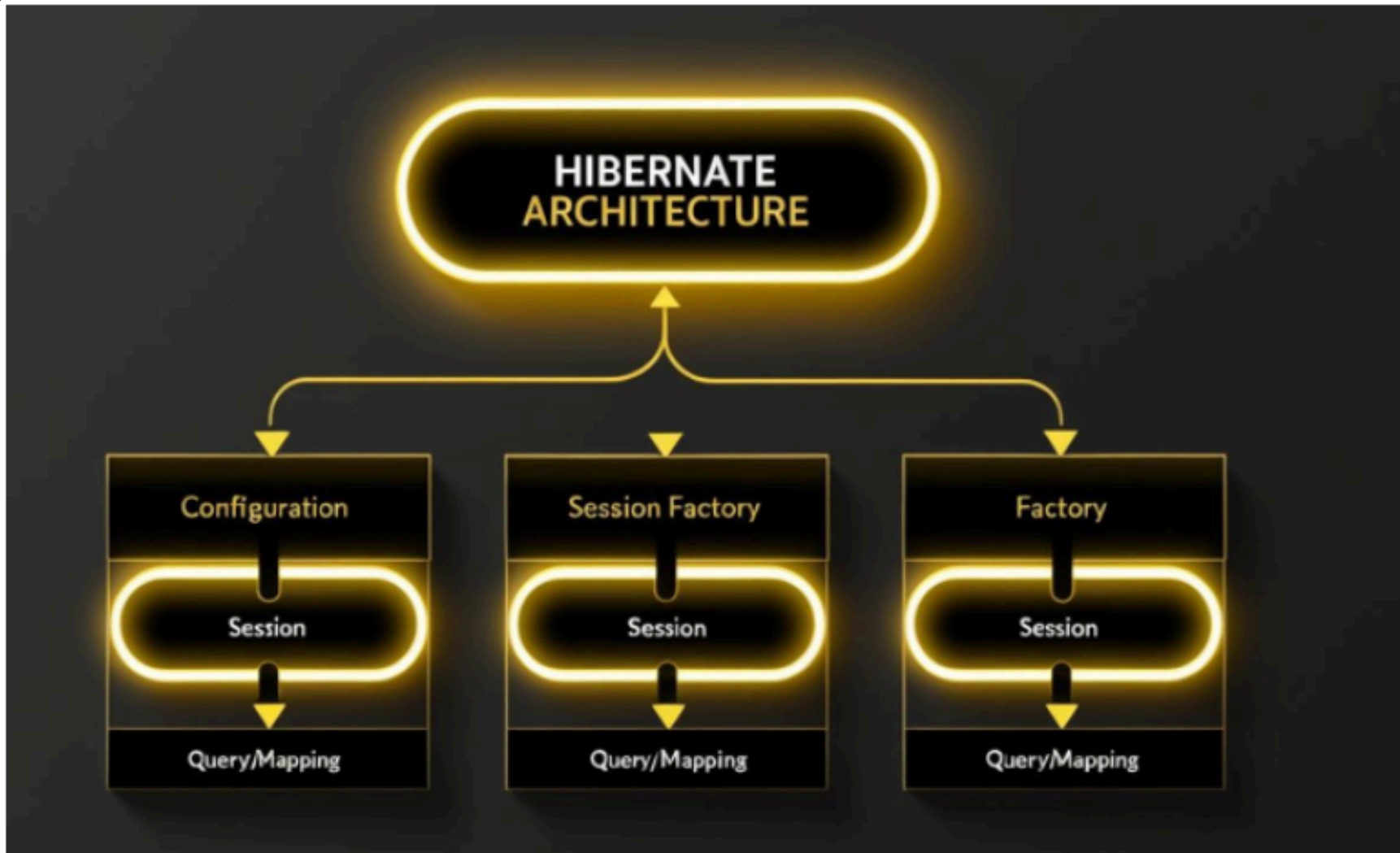
This document is strictly for personal use only and not for redistribution or commercial purposes.
If you wish to use any part of this document, you must credit the source and creator: [@javatechcommunity.com](https://javatechcommunity.com) - support@javatechcommunity.com



Join us at javatechcommunity.com for our Premium Java Interview Kit and get referrals for Java-based roles – absolutely FREE!



EXPLAIN HIBERNATE ARCHITECTURE



Hibernate is an **ORM tool for Java** that maps Java objects with database tables and allows database connections without complex SQL.

Hibernate Components: Configuration, Session Factory, Session, Transaction, Query, Mapping.

-  **Configuration:** Bootstraps Hibernate by providing database connection details and configuration options.
-  **Session Factory:** Factory interface that creates and manages sessions; a heavyweight object created once per application instance, mainly used for caching.
-  **Session:** Used to perform CRUD operations on entities.
-  **Transaction:** Manages transactions, ensuring data consistency and integrity
-  **Query:** Provides a way to execute SQL and HQL queries.
-  **Mapping:** Maps Java classes to database tables.



⚠ Copyright Notice:

This document is strictly for personal use only and not for redistribution or commercial purposes.

If you wish to use any part of this document, you must credit the source and creator: [@javatechcommunity.com](https://javatechcommunity.com) - support@javatechcommunity.com



Join us at javatechcommunity.com for our Premium Java Interview Kit and get referrals for Java-based roles – absolutely FREE!



🔥 LAZY LOADING VS EAGER LOADING IN HIBERNATE 🔥



- **Lazy Loading vs Eager Loading** are **data loading strategies** used by Hibernate. |
- While establishing **association mapping** between entities, we use the **fetch attribute** to specify the **data loading strategy**.
- It indicates **whether to load child class objects** when loading a **parent class object**.

Example: If we have a **parent class** with multiple **child classes**:

- If **fetch = LAZY** – Hibernate **loads only the parent object**, **excluding child data**
- If **fetch = EAGER** – When a **parent record is fetched**, **all child records are also loaded**.

📌 Lazy vs Eager Loading: Key Differences

Feature 📄	Lazy Loading 🐢	Eager Loading ⚡
Default Behavior	Default for @ManyToOne & @OneToMany relationships	Default for @OneToOne & @ManyToOne relationships
Data Fetching	Fetches only parent data , excludes child data unless requested	Fetches both parent & child data by default
Performance	Faster initial loading , as it fetches minimal data	Slower initial loading , as it fetches all related data
Fetch Attribute	<code>fetch = FetchType.LAZY</code>	<code>fetch = FetchType.EAGER</code>
Use Case	Best for large datasets to improve performance	Best when all data is needed upfront
Database Queries	Reduces queries by loading only required data	Increases queries but avoids extra DB hits later



⚠️ **Copyright Notice:**

This document is strictly for personal use only and not for redistribution or commercial purposes.

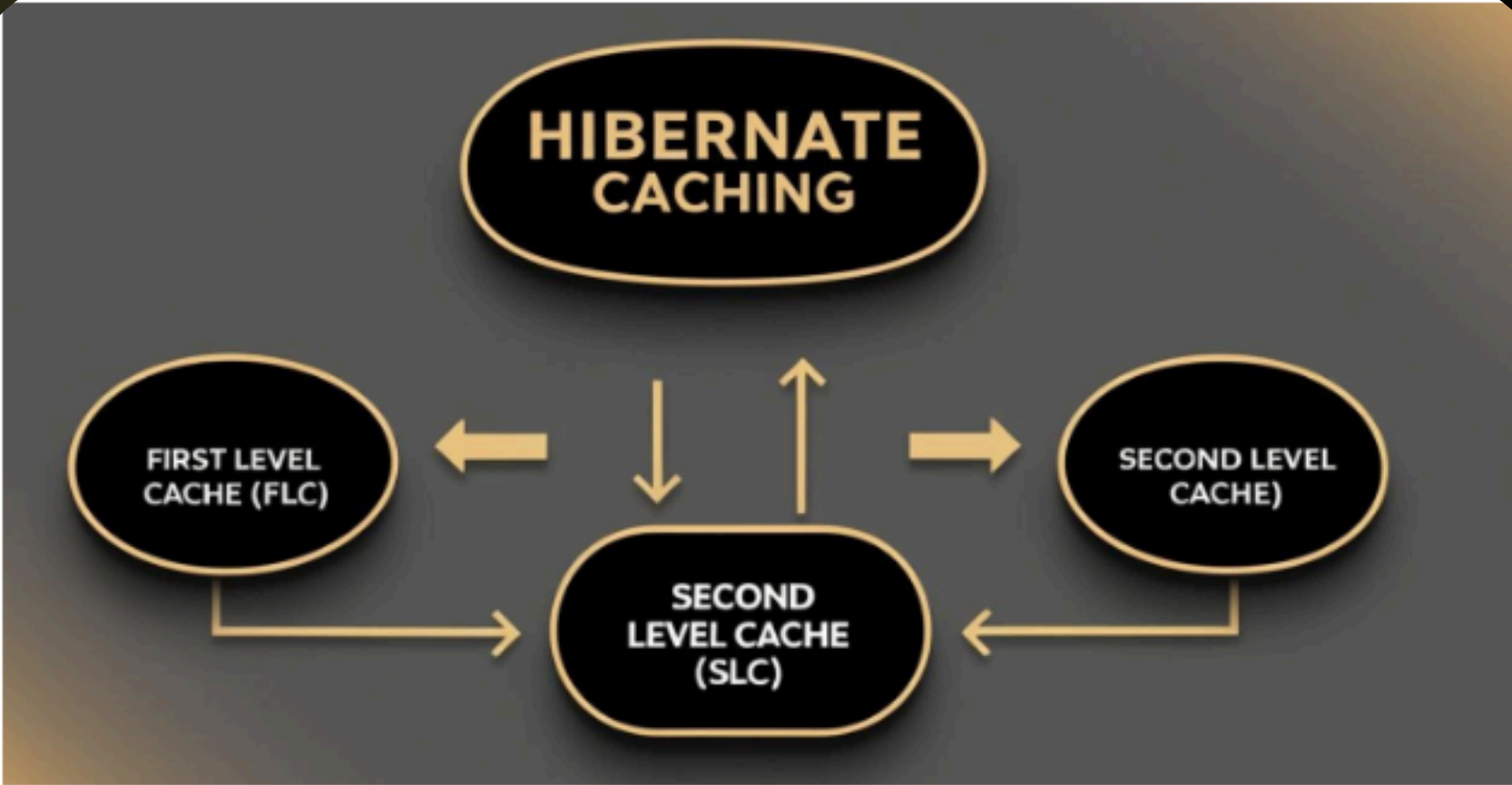
If you wish to use any part of this document, you must credit the source and creator: [@javatechcommunity.com](https://javatechcommunity.com) - support@javatechcommunity.com



Join us at javatechcommunity.com for our Premium Java Interview Kit and get referrals for Java-based roles – absolutely FREE!



WHAT IS CACHING AND HOW DOES IT WORK?



- **Caching** stores frequently accessed data in memory for **faster access**.
- **JPA** provides **First Level Cache (FLC)**, **Second Level Cache (SLC)**, and **Query Cache**.
- **Caching improves performance** and **reduces database round trips**.
- Used in the **data layer** to retrieve data efficiently.
- Best for **frequently accessed, less frequently updated** data.
- **Hibernate provides multiple caching options** to optimize queries.
- **Avoids unnecessary database queries** and speeds up application performance.

📊 First Level Cache vs Second Level Cache

Feature 🏷️	First Level Cache (FLC) 🏆	Second Level Cache (SLC) 🏅
Scope	Works within a single transaction	Shared across multiple transactions
Enabled by Default?	✅ Yes	❌ No (requires configuration)
Cleared When?	After transaction commit/rollback	Stays active across multiple sessions
Performance Impact	Avoids repeated queries within a transaction	Reduces queries across sessions/users
Configuration Needed?	❌ No	✅ Yes (needs cache provider setup)
Cache Provider	Not required (built-in)	Requires Ehcache, Infinispan, etc.



⚠️ **Copyright Notice:**

This document is strictly for personal use only and not for redistribution or commercial purposes.
If you wish to use any part of this document, you must credit the source and creator: [@javatechcommunity.com](https://javatechcommunity.com) - support@javatechcommunity.com



Join us at javatechcommunity.com for our Premium Java Interview Kit and get referrals for Java-based roles – absolutely FREE!



PURPOSE OF THIS DOCUMENT

- **Use this PDF as a quick last-minute revision guide only.**
 - **For more details, please perform Research and gain a deeper understanding on each topic.**
- **Once the interview series is complete, we will cover this topic in detail.**



⚠ Copyright Notice:

This document is strictly for personal use only and not for redistribution or commercial purposes.

If you wish to use any part of this document, you must credit the source and creator: [@javatechcommunity.com](https://javatechcommunity.com) - support@javatechcommunity.com